

Evidence is All We Need: Do Self-Admitted Technical Debts Impact Method-Level Maintenance?

1st Shaiful Chowdhury

*SQM Research Lab, Computer Science
University of Manitoba
Winnipeg, Canada*

Email: shaiful.chowdhury@umanitoba.ca

2nd Hisham Kidwai

*SQM Research Lab, Computer Science
University of Manitoba
Winnipeg, Canada*

Email: kidwaih1@myumanitoba.ca

3rd Muhammad Asaduzzaman

*School of Computer Science
University of Windsor
Windsor, Canada*

Email: masaduzz@uwindsor.ca

Abstract—Self-Admitted Technical Debt (SATD) refers to the phenomenon where developers explicitly acknowledge technical debt through comments in the source code. While considerable research has focused on detecting and addressing SATD, its true impact on software maintenance remains underexplored. The few studies that have examined this critical aspect have not provided concrete evidence linking SATD to negative effects on software maintenance. These studies, however, focused only on file- or class-level code granularity. This paper aims to empirically investigate the influence of SATD on various facets of software maintenance at the method level. We assess SATD's effects on code quality, bug susceptibility, change frequency, and the time practitioners typically take to resolve SATD.

By analyzing a dataset of 774,051 methods from 49 open-source projects, we discovered that methods containing SATD are not only larger and more complex but also exhibit lower readability and a higher tendency for bugs and changes. We also found that SATD often remains unresolved for extended periods, adversely affecting code quality and maintainability. Our results provide empirical evidence highlighting the necessity of early identification, resource allocation, and proactive management of SATD to mitigate its long-term impacts on software quality and maintenance costs.

Index Terms—SATD, bug-proneness, change-proneness, code metrics, software maintenance.

I. INTRODUCTION

Software maintenance is a crucial component of the software development lifecycle, involving activities such as bug fixing, feature enhancement, performance improvement, and perfective changes. The costs associated with software maintenance have long been a concern [1], often surpassing the original development costs [2] and accounting for approximately 70% of the total software expenditure [3]. As a result, numerous studies have focused on identifying and predicting components of code that are prone to maintenance issues, enabling proactive measures to alleviate future maintenance challenges [4], [5], [6], [7], [8], [9].

Technical debt [10], [11], [12] is a metaphor for compromises made in technical quality that yield short-term benefits but may adversely affect software quality in the long term. A common form of technical debt is self-admitted technical debt (SATD), where developers leave comments in the

source code to acknowledge suboptimal solutions, such as temporary workarounds, while signaling the need for future improvements [11], [13]. SATD is considered detrimental to future maintenance, prompting extensive research on both its detection [14], [15], [16], [17], [18], [19] and removal [20], [21], [22]. SATD is deemed so detrimental that during code review processes, code containing SATD is accepted significantly less frequently than code that is free of such debt [23]. Unfortunately, making optimizations to remove SATD often requires significant effort [24].

Despite the substantial research and effort dedicated to SATD detection and removal based on the assumption that *SATD is detrimental to maintenance*, it is surprising that these claims lack substantial validation through empirical evidence. The few studies that have investigated the impact of SATD on software quality or maintenance found no concrete evidence to support its adverse effects. For instance, a study by Wehaibi *et al.* [25] identified no clear correlation between defects and SATD. According to the authors, “*In some of the studied projects, self-admitted technical debt files have more bug-fixing changes, while in other projects, files without it had more defects.*” Similarly, Potdar and Shihab [11] evaluated the relationship between code quality and SATD and found that SATD did not affect any of the studied complexity metrics related to quality. This unexpected observation was also noted by Bavota and Russo [26]. Interestingly, although it is hypothesized that practitioners incur SATD due to time pressure, no empirical evidence was found to support this hypothesis when tested [11], [27].

This raises a crucial question: *are we investing time in developing methods for detecting and resolving SATD without any concrete evidence supporting their negative impacts?* As a scientific community, we recognize the importance of intuition in generating hypotheses and guiding exploration. However, our decision-making processes must be grounded in empirical data and rigorous evidence, rather than relying solely on intuition or anecdotal observations [28]. A compelling example is the work of Inozemtseva and Holmes [29], who critically examined the long-held belief that code coverage is

a reliable indicator of test suite effectiveness. After a thorough evaluation, the authors found that the correlation between coverage and effectiveness is not strong. This landmark paper, which won the 2024 ICSE Most Influential Paper Award, shifted the paradigm by promoting more nuanced testing strategies by balancing coverage with other relevant metrics in assessing test suite quality.

In this paper, we critically evaluate the prevailing belief regarding the negative impact of SATD, and pose the question, *is SATD truly harmful to software quality and maintenance?* Our analysis focuses on method-level granularity, which is crucial because previous studies [11], [26], [25]—which found no clear evidence of SATD’s negative impact—were conducted at the class or file level. Our motivation to study SATD at the method level is driven by three key observations:

- Practitioners find method-level maintenance tasks significantly easier than those at the file or class level [30], [5], as identifying and resolving issues at the method level typically requires much less time—methods are generally smaller than files or classes.
- Numerous studies have demonstrated no correlation between file or class-level internal software metrics and maintenance burdens [31], [32], [33], probably due to the lack of variability in measurements, as classes and files are typically large. However, these metrics have proven to be valuable indicators of maintenance when evaluated at the method level [34], [9], [4]—methods are generally much smaller than classes and files.
- Through various transformations, a method containing SATD can be relocated to a different file [30], [35]. As a result, tracking SATD at the class or file level, as done in previous studies [11], [26], [25], may lead to inaccuracies.

To gain an accurate understanding of the method-level maintenance impact of SATD, we analyze 774,051 Java methods from 49 popular open-source projects. Our investigation focuses on how SATD affects code quality, bug-proneness, and change-proneness of source code methods. Our contributions are based on the following four research questions:

RQ1: Do methods with SATD impact code quality? We evaluate 14 popular method-level code metrics, and the results are consistent across all metrics. In general, our analysis indicates that methods containing SATD exhibit significantly lower code quality, characterized by reduced readability and increased size and complexity.

RQ2: Do methods with SATD impact code change-proneness? We demonstrate that methods with SATD are more change-prone, undergoing more frequent revisions than those without SATD. This increased change-proneness indicates that SATD is a strong predictor of future maintenance challenges, leading to higher maintenance costs.

RQ3: Do methods with SATD impact code bug-proneness? Our study reveals that methods with SATD are more likely to be associated with bugs, as these methods are associated more strongly with bug-fix commits than their non-SATD counterparts. This highlights the critical importance of addressing SATD to enhance both maintainability and reliability.

RQ4: How long does it take to resolve SATD? Given the empirical evidence of the negative impact of SATD on maintenance, we aim to understand the duration required to eliminate SATD from software systems. Unlike previous studies, we approach this question with a more precise change history of SATD, facilitated by recently developed method history tracking tools, such as CodeShovel [30]. Our findings reveal that a major portion of SATDs (> 60%) is never removed from the software systems. Even when SATDs are resolved, they often persist for long periods, with many remaining unresolved for over a year. Notably, we show that 20% of SATDs are resolved after 1,000 days, a very concerning observation not reported in earlier research.

In summary, with empirical evidence, our findings underscore the detrimental effect of SATD on software maintainability, highlighting the need for early detection and remediation. To enable replication, we share our dataset publicly¹.

II. RELATED WORK AND MOTIVATION

In this section, we review previous studies on SATD detection and removal, as well as maintenance indicators, to illustrate how these studies have inspired this paper.

A. SATD Detection and Removal

Potdar and Shihab [11] created a dataset of SATD and non-SATD by manually analyzing 101,762 comments from four large open-source projects. This substantial labeled dataset is crucial for developing models to automatically detect SATD. The authors also identified 62 comment patterns that can be leveraged for automatic SATD detection. Their study has inspired numerous investigations focused on SATD detection using various machine learning models (e.g., [36], [14], [19], [15], [16]). In contrast, another stream of research has concentrated on the removal of SATD [22], [21], [20], [37], [38], [39]. However, these studies have largely operated under the assumption that SATD is detrimental to software maintenance.

B. Maintenance Impact of SATD

Surprisingly, only a limited number of studies have investigated the true maintenance impact of SATD. Zampetti *et al.* [40] utilized different code metrics and warnings from different static analysis-based tools to identify when a design-specific debt should become SATD. While they identified a relationship between code metrics and design debt, they revealed that even in the best-performing scenario, the recall for predicting design debt was only 55%. Potdar and Shihab [11] explored the effects of SATD on code quality by analyzing three key metrics: fan-in, fan-out, and cyclomatic complexity. Their findings revealed weak correlations between these metrics and SATD, with some relationships lacking statistical significance. Bavota and Russo [26] conducted a replication study involving 159 software projects, building on Potdar and Shihab’s work. They considered factors such as coupling, complexity, and code readability, yet found no correlation between code quality and SATD.

¹<https://zenodo.org/records/14058483>

Similarly, Wehaibi *et al.* [25] concluded that “*there is no clear trend when it comes to defects and self-admitted technical debt.*” Notably, they observed that changes associated with SATD tend to be more complex and are likely to introduce more future bugs compared to non-SATD changes. This raises an intriguing question: *If SATDs themselves are not inherently harmful, but addressing them may lead to greater issues, should our focus be on detecting and eliminating SATDs?*

C. Code Metrics

Internal software metrics, often referred to as code metrics, are widely used to estimate and predict external software metrics, such as bug-proneness and change-proneness [5], [4], [31], [34], [41], [7]. Unlike external metrics, internal metrics are easy to measure and can serve as reflective indicators of software quality and maintenance throughout the software development lifecycle. However, the effectiveness of code metrics in understanding maintenance has been a subject of extensive debate [9]. For instance, Gil *et al.* [31] found that, with the exception of *size*, code metrics are not particularly useful for building models that predict future maintenance burdens. This finding aligns with other similar studies [32], [33]. Conversely, Landman *et al.* [34] noted that the limited usefulness of code metrics primarily applies at the file or class level. When analyzing metrics at the method level, however, metrics like McCabe’s cyclomatic complexity [42] can serve as effective predictors of maintenance challenges. This has been further validated by multiple studies [43], [4], [9]. *These findings motivated our investigation into the code quality impact of SATD at the method-level granularity (RQ1).*

D. Bug- and Change-proneness

While the relationship between code metrics and maintenance has been debated, there is a consensus that change-proneness and bug-proneness of code components are among the most reliable indicators of software maintenance [5], [4], [44], [6], [45], [31], [46], [47]. Code components that exhibit higher bug-proneness can lead to significant future burdens, affecting both revenue and reputation. Similarly, well-designed code should require minimal revisions, in line with the *open-closed principle*—software should be open for extension but closed for modification [48]. *Consequently, we compare the change- and bug-proneness of SATD methods with that of NOT-SATD methods (RQ2 and RQ3).*

E. SATD Removal Time

Multiple studies have examined the duration it takes to remove SATDs from software projects. Potdar and Shihab [11] noted that up to 75% of SATDs were never removed within the projects they studied. In some cases, only about 37% of SATDs were addressed by the next release. This indicates that SATDs can persist in software projects for extended periods. This finding was further supported by Maldonado *et al.* [22], who reported that it may take 18 to 172 days (on median) to remove SATDs. Conversely, Liu *et al.* [38] found that SATDs are generally removed more quickly than Maldonado

et al. suggested. However, these studies did not effectively track SATD, particularly when methods containing SATD were moved or renamed. Generally, previous approaches treated SATD as removed when the containing method or file could no longer be found. With the advent of advanced method tracing tools like CodeShovel [30] and CodeTracker [35], we can now accurately trace methods even after they have been renamed or relocated. *Therefore, with a new more accurate method-level dataset, we aim to re-investigate this issue: what percentage of SATDs are typically removed, and how long do the SATDs generally last in software projects (RQ4)?*

III. METHODOLOGY

In this section, we outline our project selection criteria, methods for collecting change history, and SATD detection. We also cover the statistical tests utilized in our study. To enhance readability, methods unique to specific questions are addressed in their respective sections in Section IV.

A. Project Selection

To explore the influence of SATD on method-level maintenance, we selected a range of open-source software projects from GitHub², a leading platform for mining software repositories. Previous MSR-based research has typically taken one of two approaches: aggregated analysis [31], [5], [7] or individual project analysis [1], [6], [41], [8]. In aggregated analysis, the results are presented as a summary of all projects combined, rather than individually for each project. While this approach can be useful, it is vulnerable to the influence of outliers, potentially skewing findings. On the other hand, individual project analysis provides a more granular view, and we can observe how SATD affects different projects. This approach, however, is prone to issues such as selection bias or publication bias [49], [31].

To address these limitations, we conducted our study using both approaches on a selected set of 49 open-source Java projects—the tools we used, e.g., CodeShovel, are compatible only with Java, which limited our ability to analyze projects based on other programming languages. We present results aggregated across these projects, in addition to providing more detailed insights from individual project analyses. Also, rather than selecting projects at random, we selected all the projects used in five MSR-based studies [31], [30], [47], [50], [7], ensuring a broad and representative dataset. This approach allows us to draw more robust and widely applicable conclusions about the role of SATD in open-source software maintenance. Our analysis begins with a total of 774,051 open-source methods, as detailed in Table I.

B. Tracking Change History

Tracking changes at the method level is inherently difficult due to factors such as renaming, reparameterizing, or relocating methods across different files [30]. To overcome this challenge, we utilized CodeShovel [30], a tool designed to trace method histories even in the face of these transformations.

²<https://github.com>

TABLE I: We selected 49 projects for our study. These projects are popular within the open-source community, as indicated by their large number of contributors and stars. The percentage of SATD methods is also presented for each project.

Repository	# Methods	# SATD Methods (%)	# Contributors	# Stars	Snapshot
hadoop	70081	1224 (1.75)	592	14500	4c5cd7
elasticsearch	62190	1372 (2.21)	1932	68400	92be38
flink	38081	5111 (13.42)	908	16764	261e72
lucene-solr	37133	1681 (4.53)	234	4400	b457c2
presto	36715	816 (2.22)	734	15700	bb20eb
docx4j	36514	2452 (6.72)	37	1620	36c378
hbase	36274	5853 (16.14)	463	5200	3bd542
intellij-community	35950	767 (2.13)	1035	16800	cdf2ef
weka	35639	513 (1.44)	2	323	a22631
hazelcast	35265	426 (1.21)	340	6000	a59ad4
spring-framework	26634	339 (1.27)	544	43641	1984cf
hibernate-orm	24800	930 (3.75)	420	4678	2c12ca
eclipseJdt	22124	1061 (4.80)	133	146	475591
guava	20757	491 (2.37)	264	41775	e35207
sonarqube	20627	249 (1.21)	137	5937	6b806e
jclouds	20358	182 (0.89)	231	384	7af4d8
wildfly	19665	645 (3.28)	343	2593	f21f5d
netty	16908	316 (1.87)	526	27185	662e0b
cassandra	15953	2217 (13.90)	447	8600	7cdad3
argouml	12755	3414 (26.77)	4	238	fcbe6c
jetty	10645	422 (3.96)	159	3160	fc5dd8
voldemort	10601	189 (1.78)	65	2475	a7dbde
spring-boot	10374	29 (0.28)	828	56378	199cea
wicket	10058	243 (2.42)	83	570	e3f370
ant	9781	187 (1.91)	58	303	1ce1cc
jgit	9548	208 (2.18)	140	983	855842
mongo-java-driver	9467	109 (1.15)	151	2422	8ab109
pmd	8992	282 (3.14)	229	3482	d115ca
xerces2-j	8153	289 (3.54)	3	38	cf0c51
RxJava	8145	145 (1.78)	278	44922	880eed
openmrs-core	6066	184 (3.03)	371	1017	c5928a
javaparser	5862	110 (1.88)	157	3771	8f25c4
hibernate-search	5345	212 (3.97)	58	390	5b7780
titan	4590	103 (2.24)	34	5135	ee226e
facebook-android-sdk	3759	69 (1.84)	124	6100	fb1b91
checkstyle	3340	89 (2.66)	287	6115	164a75
commons-lang	2948	58 (1.97)	160	2138	f69235
lombok	2684	78 (2.91)	114	10391	4fdcd8
atmosphere	2659	77 (2.90)	111	3524	fadfb0
jna	2636	54 (2.05)	143	6655	b8443b
Essentials	2390	52 (2.18)	215	1086	d36d80
junit5	2085	34 (1.63)	167	4691	be2aa2
hector	1958	53 (2.71)	71	648	a302e6
okhttp	1953	144 (7.37)	236	40428	5224f3
mockito	1498	107 (7.14)	21	12043	77562
cucumber-jvm	1146	26 (2.27)	226	2291	b57b92
commons-io	1145	23 (2.01)	76	787	11f0ab
vraptor4	926	9 (0.97)	48	343	593ce9
junit4	874	67 (7.67)	151	8160	50a285
Total	774051	33711 (4.36)			

In both precision and recall, CodeShovel outperformed other tools including FinerGit [51], `git log` [30], and IntelliJ’s *show history for selection* [30] approach. CodeShovel also exhibited high accuracy in independent evaluations by industry developers. Although a more recent tool, CodeTracker [35], was shown to be more accurate than CodeShovel on a researchers-made oracle, CodeTracker was not independently validated with industry practitioners. Also, the run-time of CodeTracker is slower than CodeShovel, making it difficult to use for collecting the change history of 774,051 methods that we used. CodeShovel has been used successfully to perform method-level maintenance analysis (e.g., [4], [9], [52]).

CodeShovel offers an advanced method tracking system within Git repositories, enabling the capture of critical details about when, why, and how methods are altered over time. The output of CodeShovel is one JSON file for each method, containing all the change commits along with the different versions of source code and other important information. We developed a custom tool that interacts with CodeShovel, enabling us to retrieve comprehensive historical data and different code and change metrics for each method. The tool leveraged the popular JavaParser³ library to parse the source

³<https://github.com/javaparser/javaparser> last accessed: Sep 20, 2024

code and generate abstract syntax trees (ASTs) that served as the primary source of code metric calculations. This tool was independently validated by two different researchers with 1000 randomly sampled Java methods. In all cases, the tool correctly measured different code metrics and change histories from the CodeShovel's output.

C. Detecting SATD in Methods

The objective of our study is to evaluate if SATD methods are more maintenance-prone than NOT-SATD methods. This required us to collect all the code comments contained in each of the studied methods. For a given method, we fed all of its comments to *SATD detector*, a text-mining-based tool developed by Liu *et al.* [14]. We selected this tool because of its availability as an easy-to-use `jar` file and its adoption in other SATD studies [38], [40], [53]. A source method was labeled as a method with SATD if at least one of its comments is identified as SATD by the *SATD detector* tool.

D. Visualization, Statistical Tests, and Design Choices

To compare various code quality and maintainability indicators between methods with and without SATD, we extensively use the Cumulative Distribution Function (CDF). Unlike the Probability Distribution Function (PDF), the CDF is monotonic, which makes it easier to interpret the data by avoiding irregular or zigzag patterns. Additionally, we aim to assess whether the visual differences between the SATD and NOT-SATD groups are statistically significant.

Using the Anderson-Darling normality test [54], we found that the distributions of our data did not follow a normal distribution. As a result, we employ the non-parametric Wilcoxon rank sum test to assess statistical significance. When we identify significant differences between distributions, we quantify the size of the effect using Cliff's d , a non-parametric measure of effect size. Cliff's d ranges in the interval $[-1, 1]$ and is considered *negligible* for $|d| < 0.147$, *small* for $0.147 \leq |d| < 0.33$, *medium* for $0.33 \leq |d| < 0.474$, and *large* for $|d| \geq 0.474$. Both the Wilcoxon rank-sum test and Cliff's delta are widely used in data-driven software engineering research [55], [56], [57], [58], [59]. Similar to other relevant studies [29], [31], [58], when calculating correlation coefficients, we used the non-parametric Kendall's τ instead of the Pearson correlation coefficient.

The methodological choices made by researchers in software maintenance studies impact the outcomes of their analysis [52]. During our investigation into SATD-labeled methods and various maintenance indicators, we encountered two possible approaches for classification and analysis:

- In the first approach, a method is labeled as a SATD method if it contains at least one SATD comment in any of its versions—ranging from its initial introduction to the most recent updates. Maintenance indicators (e.g., size, readability) can then be calculated based on different time points, such as the method's introduction, its most recent state, or the mean or median values over its history.

- In the second approach, a method is labeled as SATD if it contains a SATD comment in its initial version. Subsequent maintenance indicators are calculated from later versions of the method. To qualify as a NOT-SATD method, the method must not contain any SATD comments throughout its entire lifetime.

After thoroughly testing both approaches, we found that they did not substantially affect the study's results. We opted to present the findings based on the second approach, as it better aligns with the main objective of our research: it allows us to examine whether methods that start with SATD comments become more maintenance-prone over time.

IV. APPROACH, ANALYSIS, AND RESULTS

We now present the approach used to address each of the four research questions, along with the corresponding findings.

A. RQ1: Do methods with SATD impact code quality?

Code metrics have been historically used as the reflective indicators of code quality and software maintenance [31], [5], [4]. As such, we study the impact of SATD on code metrics measurements at the method level. We focus on 14 different code metrics from the following six broad categories.

Size. Size has been considered one of the most important code metrics to understand software maintenance burdens [31], [60], [32]. We calculated size as the number of source lines of code without comment and blank lines, in accordance with similar previous studies [34], [4], [49].

Readability. A core activity in software maintenance is code reading [61]. Code readability refers to how easily a developer can comprehend the structure and logic of the source code [62], [63], [64], [65]. High readability is essential for efficient maintenance, as it reduces the cognitive load on developers and accelerates the process of updating and fixing code. In this study, we employed two distinct readability metrics: one proposed by Buse *et al.* [62] (*Readability*) and the other by Posnett *et al.* [63] (*SimpleReadability*). Both metrics assign a readability score to each method, ranging from 0 (least readable) to 1 (most readable), providing a quantitative measure of how easily the code can be understood.

Testability. We utilize the McCabe metric [42], or cyclo-matic complexity, to assess the testability of source methods. This metric quantifies the number of independent execution paths within a method, which serves as an indicator of its testing complexity. However, McCabe's metric does not account for the number of control variables or comparison operations within a method's predicates. As such, we also incorporate two additional factors: the number of control variables (*NVAR*) and the number of comparisons (*NCOMP*), which contribute to a method's overall complexity. Additionally, we include the McClure metric [66], which is calculated as the sum of *NVAR* and *NCOMP*. Calculating McCabe and McClure metrics typically requires a language-specific parser. Hindle *et al.* [67] proposed proxy indentation as an alternative, which is also included in our study (*IndentSTD* [9]). Finally, recognizing

that deep nesting can complicate testing, we also measured the Maximum Nested Block Depth.

Dependency. If a method depends on too many other methods, any problems with those methods can be propagated to the caller method. Dependency, therefore, is an important quality and maintenance indicator. We calculated the total number of methods called by a given method (*totalFanOut*) as its dependency measurement.

Maintainability. The Maintainability Index is a composite metric that combines various code metrics to provide a single score reflecting the maintainability of a software component. It is calculated as follows:

$$171 - 5.2 * \ln(Halstead V) - 0.23 * (McCabe) - 16.2 * \ln(Size) \quad (1)$$

We include this index in our study because of its widespread adoption in industry-standard tools, such as Verifysoft Technology⁴ and Visual Studio⁵.

Other. We also include Halstead Length, number of parameters, and number of local variables to answer this research question. We could not include the fan-in metric that we discuss in the threats to validity section.

1) Results: Figure 1 compares the distributions of six code metrics between SATD and NOT-SATD methods. To maintain clarity, we omit graphs for other metrics, though all of them exhibit patterns similar to the first row, with SATD methods showing larger values than NOT-SATD methods. These metrics were measured from the most recent versions of the methods to assess how SATD methods evolve compared to NOT-SATD methods. The results reveal a clear trend: SATD methods grow larger, less readable, and less maintainable over time. For instance, Figure 1 (a) shows that 60% of SATD methods have more than 10 lines of code, while only 20% of NOT-SATD methods exceed this threshold. Similarly, the MaintainabilityIndex is above 100 for 80% of NOT-SATD methods, compared to just 40% for SATD methods.

We further investigate whether these differences are statistically significant and, if so, we calculate their effect sizes. The results for all 14 code metrics are shown in Table II. The P-values from the Wilcoxon rank-sum test are all ≤ 0.05 , indicating that the distributions of all code metrics are statistically significant. We also applied the Benjamini-Yekutieli correction approach [68] and found no noticeable difference in the results. Additionally, none of the effect sizes are small or negligible, except one; most are large, with a few exhibiting medium effect sizes. For three code metrics, marked with a '-' sign, the measurements for SATD methods are smaller than for NOT-SATD methods, reinforcing our earlier finding that SATD methods are less readable and maintainable.

To assess the generalizability of our results, we also conduct an analysis at the project level. The findings are summarized in Table III. Consider the *Readability* metric as an example.

⁴https://verifysoft.com/en_maintainability.html: last accessed: Sep-28-2024

⁵<https://docs.microsoft.com/en-us/visualstudio/code-quality/code-metrics-maintainability-index-range-and-meaning?view=vs-2022>: last accessed: Sep-28-2024

We first determine the percentage of projects where the distribution difference between SATD and NOT-SATD methods is not statistically significant (i.e., $P > 0.05$). For projects where a statistical difference is found, we categorize the effect sizes as negligible, small, medium, or large. For most code metrics, differences between SATD and NOT-SATD methods are statistically significant across the majority of projects, with effect sizes predominantly falling between medium and large. The few projects that show no meaningful differences are those with a smaller number of SATD methods in our dataset, which limits the statistical power of the analysis.

TABLE II: Aggregated analysis for all the code metrics. The P - values remain significant even after applying the Benjamini-Yekutieli correction approach [68].

Metric	P	Sign	Effect Size
Size	0.00	+	large
Readability	0.00	-	medium
SimpleReadability	0.00	-	large
NVAR	0.00	+	medium
NCOMP	0.00	+	medium
Mcclure	0.00	+	medium
McCabe	0.00	+	large
IndentSTD	0.00	+	medium
MaximumBlockDepth	0.00	+	large
totalFanOut	0.00	+	large
Length	0.00	+	large
MaintainabilityIndex	0.00	-	large
Parameters	0.00	+	small
LocalVariables	0.00	+	large

TABLE III: Individual project analysis for all code metrics. The P - values remain very similar after applying the Benjamini-Yekutieli correction. Here, **N** refers to Negligible, **S** refers to Small, **M** refers to Medium, and **L** refers to Large effect size. For instance, for the size metric, the effect size is large for 81.63% of projects.

Metric	P > 0.05	N	S	M	L
Size	0.00	0.00	4.08	14.29	81.63
Readability	6.52	2.17	8.70	43.48	45.65
SimpleReadability	2.08	0.00	8.33	20.83	70.83
NVAR	2.08	2.08	12.50	33.33	52.08
NCOMP	2.08	2.08	10.42	35.42	52.08
Mcclure	2.08	2.08	10.42	35.42	52.08
McCabe	4.26	0.00	10.64	25.53	63.83
IndentSTD	0.00	0.00	12.24	30.61	57.14
MaximumBlockDepth	4.26	0.00	10.64	27.66	61.70
totalFanOut	2.08	0.00	4.17	18.75	77.08
Length	0.00	0.00	2.04	8.16	89.80
MaintainabilityIndex	0.00	0.00	2.04	10.20	87.76
Parameters	28.95	2.63	63.16	34.21	0.00
LocalVariables	4.26	0.00	10.64	21.28	68.09

RQ1 summary: Building on the longstanding assumption that code metrics are reliable indicators of software quality and maintainability, our findings confirm that SATD in source code significantly degrades both. Methods with SATD tend to grow larger, more complex, less readable, and less maintainable compared to methods that have never contained SATD.

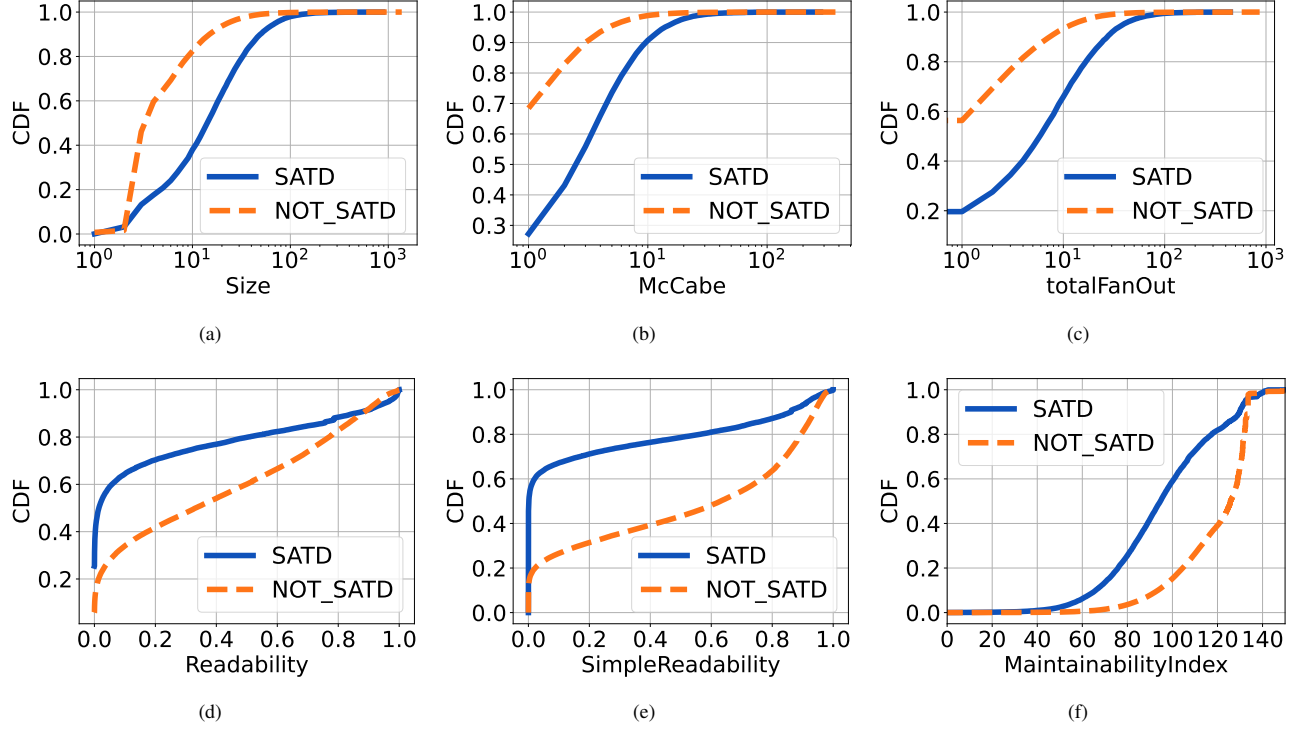


Fig. 1: Differences in distribution for Size, McCabe, totalFanOut, Readability, SimpleReadability, and MaintainabilityIndex in the aggregated data. Clearly, SATD methods tend to have larger sizes, more complexity, more dependency, and lower readability, and worse Maintainability Index compared to NOT-SATD methods.

B. RQ2: Do methods with SATD impact change-proneness?

Our previous analyses of code metrics have shown that SATD negatively impacts code quality and maintenance. However, there has been an ongoing debate in the literature regarding the true effectiveness of code metrics in understanding software maintenance challenges [9]. For instance, while some studies have endorsed the utility of McCabe’s cyclomatic complexity [69], [34], [70], [8], [71], others have questioned its effectiveness [72], [73], [31]. This skepticism extends to other widely-used code metrics as well [31].

Given these varying perspectives, we also focus on two well-established maintenance indicators—change- and bug-proneness—since the software engineering community generally agrees on their reliability for assessing maintenance burdens [74], [75], [46], [76], [77], [6], [4], [5], [43]. In this RQ, we investigate how SATD influences the future change proneness of source methods. Specifically, we examine whether methods that start with SATD become more change-prone than methods that have never contained SATD. Encouraged by relevant previous studies, we selected the following five change-proneness indicators.

#Revisions. The number of revisions a code component undergoes is often used as an indicator of its quality and maintainability, under the assumption that better-designed components require fewer modifications [78], [79], [41], [80],

[43], [5]. To calculate the revision count for a method, we relied on CodeShovel’s recorded change commits, excluding those where no actual code was altered—when a method is relocated or a file is renamed without changing its contents.

DiffSizes and NewAdditions. The number of revisions alone does not fully reflect a component’s change-proneness, as revisions can vary greatly in terms of the effort required—some may involve minimal changes, while others can be more extensive. To address this, diff size is often used as a more comprehensive measure [43], [81], [41]. However, since adding new code is typically more challenging than removing it, the count of added lines (NewAdditions) is sometimes considered a better indicator of change-proneness [41], [43], [5]. We calculated the DiffSizes by summing the diff sizes for each change commit. NewAdditions were computed in a similar fashion. This approach allowed us to capture the full scope of changes made to a method throughout its lifecycle, providing a more accurate measure of its evolution.

EditDistances and CriticalEditDistances. DiffSizes and NewAdditions fail to account for the differences in the length of modified lines. To overcome this limitation, we employed the Levenshtein edit distance [82], [83], [84], [81], which quantifies the number of insertions, deletions, or substitutions required to convert one version of a method to another. Similar to DiffSizes and NewAdditions, we calculated the

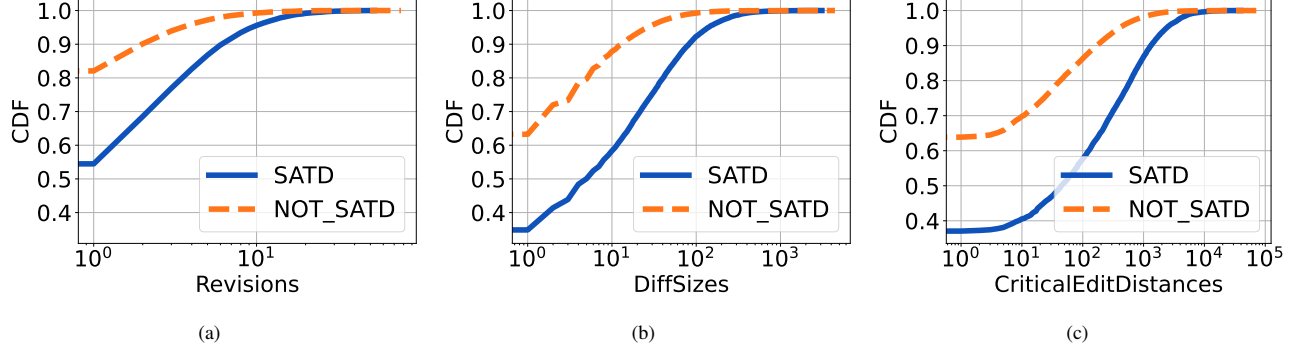


Fig. 2: Differences in distribution for #Revisions, DiffSizes, and CriticalEditDistances. Clearly, SATD methods tend to have more revisions and larger change sizes. The results are very similar for NewAdditions and EditDistances, therefore, not presented.

EditDistances by summing the edit distances for each change commit. CriticalEditDistances were computed in a similar manner, with the key difference that non-critical changes—such as whitespace modifications and comment updates—were excluded. This adjustment was necessary, as previous research has shown that such non-critical changes can skew the true impact of code modifications [52].

1) *Results*: Figure 2 illustrates that SATD methods experience more revisions and exhibit significantly larger change sizes compared to NOT-SATD methods. For example, more than 45% of the SATD methods were revised at least once, which is true only for 20% of NOT-SATD methods. Table IV further confirms that SATD methods are statistically distinct from NOT-SATD methods across all change-proneness metrics ($P \leq 0.05$). Additionally, for all change-proneness indicators, SATD methods show medium effect sizes, with more revisions and larger change sizes, as indicated by the ‘+’ sign.

TABLE IV: Statistical Analysis for the aggregated data.

Indicator	P	Sign	Effect Size
Revisions	0.00	+	medium
DiffSizes	0.00	+	medium
NewAdditions	0.00	+	medium
EditDistances	0.00	+	medium
CriticalEditDistances	0.00	+	medium

TABLE V: Individual project analysis. **N** refers to Negligible, **S** refers to Small, **M** refers to Medium, and **L** refers to Large effect size. For instance, for the Revisions metric, the effect size is large for 61.70% percent of projects.

Indicator	P > 0.05	N	S	M	L
Revisions	4.26	0.00	8.51	29.79	61.70
DiffSizes	2.08	0.00	6.25	20.83	72.92
NewAdditions	4.26	0.00	6.38	27.66	65.96
EditDistances	0.00	0.00	4.08	26.53	69.39
CriticalEditDistances	4.26	0.00	8.51	23.40	68.09

We also investigated whether these change-proneness patterns vary across different projects. Table V presents the results

for all projects. For every project, the EditDistances between SATD and NOT-SATD methods are statistically significant. In fact, the few projects where no statistical differences were observed for other metrics were primarily due to the limited number of SATD samples in those projects. Notably, the majority of the projects exhibit medium to large effect sizes.

Multiple studies [9], [52] have shown that comparing methods of different ages when analyzing change- and bug-proneness can lead to inaccurate conclusions, due to the strong correlation between age and the likelihood of changes (or bugs). Older methods have had more time to evolve than newly introduced ones. Following the recommendation by Chowdhury *et al.* [9], we applied a 2-year age normalization to assess whether it affected our findings. Specifically, based on the suggestion of Chowdhury *et al.* [9], we excluded methods younger than 2 years and discarded any changes that occurred after the two-year mark. However, our analysis of change-proneness remained consistent, even after this age normalization.

RQ2 summary: Our findings, which take into account the number of revisions and various change sizes, clearly show that SATD methods are significantly more change-prone than NOT-SATD methods.

C. RQ3: Do methods with SATD impact code bug-proneness?

Bugs or defects in software are a major concern for stakeholders, often leading to significant challenges in maintenance and reliability. Given the importance of bug detection, the research community has extensively explored bug prediction and localization, with over 100,000 results on Google Scholar for the term *bug prediction*. In this study, we investigate whether SATD methods are more likely to become bug-prone in the future compared to NOT-SATD methods. For each of the 49 projects, we calculated the ratio of bug-prone methods separately for SATD and NOT-SATD methods. This allows us

to compare the bug-proneness between the two sets both at the aggregated level and for each project individually.

A method is considered bug-prone if it is associated with a bug-fix commit at a later time. While it is common to use commit messages containing bug-fix keywords to identify bug-prone components [43], [5], [4], [44], this approach is imprecise due to the issue of tangled changes. Unfortunately, tangled changes—unrelated modifications that are committed together—are common in the real world [85], [4], [86]. To mitigate the impact of tangled changes on our analysis, we conduct our analysis on three different types of bug datasets.

- **HighRecall Dataset.** A method is a bug-prone method if it is modified in a commit containing one of the following nine keywords (or their variations): *error*, *bug*, *fix*, *issue*, *mistake*, *incorrect*, *fault*, *defect*, and *flaw*. This set of keywords has been used in numerous studies [50], [87]. While this method is effective at capturing a broad range of buggy methods (resulting in high recall), it often leads to a significant number of false positives due to the challenges posed by tangled code changes.
- **HighPrecision Dataset.** To reduce false positives, Chowdhury *et al.* [4] refined the bug labeling approach by removing the keyword *issue* and adding *misfeature* based on the recommendation of Rosa *et al.* [88]. They also included fix-related terms (*fix*, *address*, *resolve*) to improve precision and excluded commits modifying more than one method to avoid tangled changes. This approach enhances precision but sacrifices recall by excluding many potential bug-fix commits.
- **Balanced Dataset.** We created this dataset using the same keyword set as the HighPrecision dataset, but with a key difference: instead of limiting bug-fix commits to modifications of a single method, we allowed up to five method modifications per commit. This adjustment aimed to strike a balance between precision and recall.

1) *Results:* Table VI presents the bug ratios for SATD and NOT-SATD methods based on the aggregated data. As expected, the HighRecall dataset shows higher bug ratios in both sets compared to the other two datasets. However, in all three datasets, SATD methods consistently exhibit a significantly higher bug ratio than their NOT-SATD counterparts.

Our findings from the aggregated data hold true across individual projects as well. Figure 3 illustrates the distribution of bug ratios across all 49 projects for each of the three datasets. For instance, in the HighRecall dataset, SATD methods have a bug ratio exceeding 0.4 in 80% of the projects. In contrast, only 18% of the projects show a bug ratio greater than 0.4 for NOT-SATD methods. Table VII confirms that all visual differences in Figure 3 are statistically significant ($P \leq 0.05$), with effect sizes ranging from large to small. The ‘+’ sign further indicates that higher bug ratios are typically associated with SATD methods.

TABLE VI: Bug ratio in aggregated data.

Bug Type	SATD methods	NOT-SATD methods
HighRecall	0.396	0.213
Balanced	0.035	0.011
HighPrecision	0.012	0.0046

TABLE VII: Statistical Analysis for Individual Projects. The small effect size with the HighPrecision dataset is due to the smaller number of bug-prone methods in both SATD and NOT-SATD sets.

Bug Type	P	Sign	Effect Size
HighRecall	0.00	+	large
Balanced	0.00	+	medium
HighPrecision	0.00	+	small

RQ3 summary: SATD methods are significantly more bug-prone than NOT-SATD methods, making them a significant liability in terms of software maintainability and reliability, increasing both cost and effort and harming reputation.

D. RQ4: How long does it take to resolve SATD?

In RQ1, RQ2, and RQ3, we have demonstrated that SATD negatively impacts software quality and maintainability. This naturally leads to the question: *how often are SATDs removed in real-world software projects, and what is the typical duration for their resolution?* While previous studies have explored these questions [22], [38], they yielded significantly different results. Additionally, prior approaches considered an SATD removed if a method containing the comment was not found in a later phase. However, this is inaccurate, as methods can be renamed or moved to a different class or file. With the introduction of CodeShovel, which we used for change tracking, we are now able to capture such transformations, resulting in a more accurate dataset for tracking SATD removal.

1) *Results:* To ensure that newer SATD methods were not unfairly classified as unresolved, we excluded SATD methods younger than 2 years from our analysis, allowing them time to evolve and potentially address their SATDs. Nevertheless, our analysis of the aggregated data showed that over 61% of SATDs were never removed (a similar observation was made by Zampetti *et al.* [21]). The percentage of unresolved SATDs, however, varies considerably across different projects. Figure 4 (a) shows the distribution of unresolved SATDs for all 49 projects. In 80% of the projects, unresolved SATDs account for more than 20%. Additionally, 20% of the projects have unresolved SATDs exceeding 60%, with a few projects showing more than 80% unresolved SATDs.

For the resolved SATDs, we also calculated their removal times across the studied software projects. Figure 4 (b) illustrates the distribution of removal times for all resolved SATDs in our dataset (aggregated data). We observe that 60% of the SATDs took at least 100 days to be resolved, while 20% required more than 1000 days to be resolved.

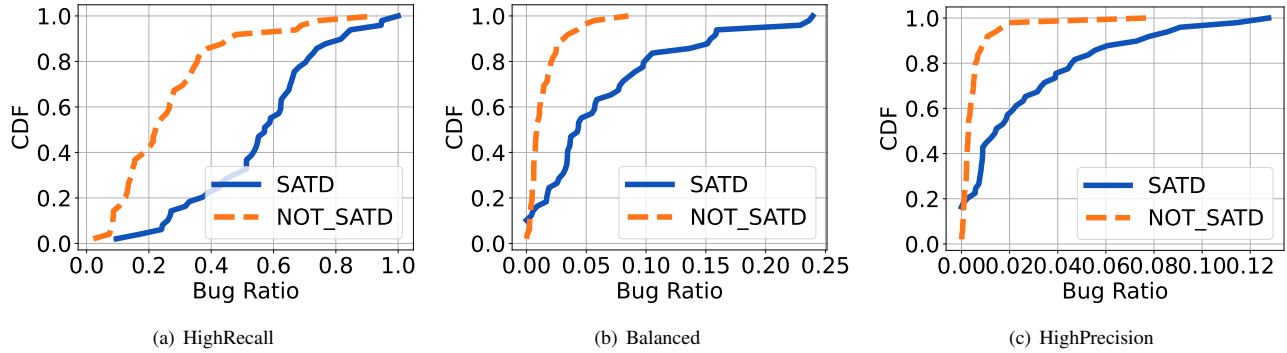


Fig. 3: Differences in the distribution of bug ratios in three datasets: SATD methods tend to have more bugs than NOT-SATD methods in all datasets.

To present the results for each project individually, we need to display 49 separate CDFs—one for each of the 49 projects—similar to the graph shown in Figure 4 (b). To simplify, we computed the mean and median SATD removal times for each project. The distributions of these means and medians are shown in Figures 4(c) and 4(d), respectively. The results reveal significant variation in SATD removal times across projects. While some projects resolve SATDs relatively quickly, others take considerably longer, with mean and median removal times differing markedly between projects. For instance, approximately 10% of projects have a median removal time of less than 10 days, while 40% of projects experience a median removal time exceeding 100 days.

RQ4 summary: A staggering ~61% of the SATDs are never removed from real-world software projects. When the SATDs are resolved, it may take more than 1000 days. While SATDs negatively impact all projects, the extent of their effect varies significantly between projects.

V. DISCUSSION

Previous research has found no strong evidence supporting the harmful effects of Self-Admitted Technical Debts, yet considerable effort has been devoted to detecting and minimizing SATDs. In light of this, we conducted a comprehensive study to assess whether SATDs genuinely impact software quality and maintainability. Unlike prior studies [11], [26], [25], which focused on class/file level analysis, our research focused on method-level granularity—a level often preferred by both practitioners and researchers [30], [5].

Our findings demonstrate that SATDs are indeed detrimental to software quality and maintenance when assessed using commonly accepted maintenance and quality indicators. Specifically, we observed that methods containing SATDs tend to be larger, more complex, less readable, and harder to maintain (**RQ1**). When examined through popular metrics,

SATD methods were found to be significantly more change-prone (**RQ2**) and bug-prone (**RQ3**), thereby escalating future maintenance burdens. Furthermore, our study reveals that the negative impact of SATDs can persist for extended periods, with many SATDs remaining unresolved (**RQ4**).

Implications for practitioners. The cost of software maintenance has been a long concern for industry practitioners [1], [2]. Our study provides strong evidence that SATDs have a significant negative impact on software quality and maintenance. Therefore, practitioners should prioritize the detection and resolution of SATDs. By addressing SATDs early, teams can mitigate the long-term maintenance burden and reduce the risk of bugs and costly rework. SATDs, especially those that remain unresolved over time, may lead to sustained negative impacts on software maintainability. Practitioners should account for these long-term costs when making technical decisions. Rather than postponing the resolution of SATDs, it may be more cost-effective to address them promptly to avoid accumulating technical debt that could hinder future development or result in harder-to-fix bugs. Tools and metrics that evaluate software quality (e.g., code complexity, change-proneness, and bug-proneness) should consider including SATD as one of the indicators. This would allow practitioners to prioritize code segments containing SATDs, thus aligning technical debt management with the software’s overall quality goals.

Implications for researchers. In this paper, we treated all types of SATDs as a single, undifferentiated category. However, previous research has shown that SATDs can be classified into several distinct types, including design, defect, documentation, requirement, and test debt [16]. Future research can replicate our study separately for each of these types and can answer whether these different SATD types impact software quality and maintenance efforts differently. Understanding these differences will enable developers to prioritize which types of SATDs require more immediate attention and which can be addressed with more flexibility. Additionally, such insights would allow researchers to develop targeted strategies for managing each type of SATD more effectively, optimizing the approach to technical debt resolution.

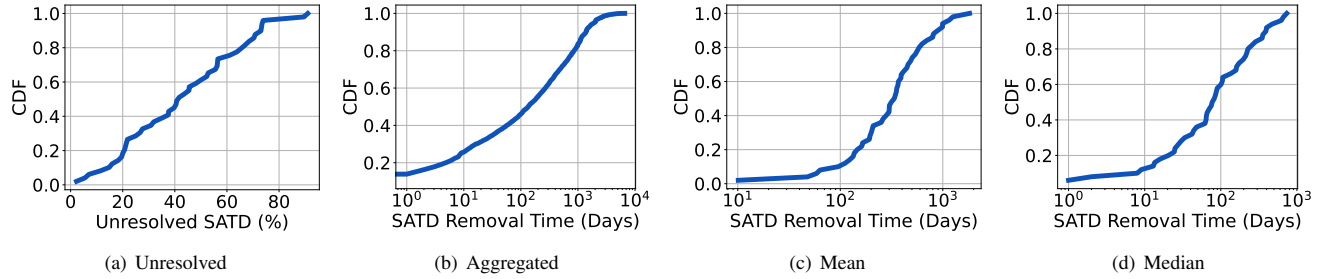


Fig. 4: Figures (a) and (b) show the distribution of the percent of unresolved SATD methods and the distribution of SATD removal time in aggregated data, respectively. Figures (c) and (d) show the distribution of the means and medians of SATD removal time for all 49 projects.

Given the strong negative impact of SATDs, our study should encourage the research community to build automated tools that can detect SATDs more accurately and provide suggestions for remediation. Research into such tools could help streamline the SATD management process, making it easier for practitioners to identify and address problematic code before it leads to more serious maintenance issues.

Moreover, based on our findings, researchers should consider incorporating SATD as a feature in machine learning models for bug and change prediction, if they have not already done so. We found that SATD has not been used (or underutilized) in such existing models [5], [43], [4], [46], [76].

A. Threats to Validity

External validity is limited due to the selection of software projects in this study. The results from open-source software may not reflect the characteristics of closed-source software, and the findings may not be directly applicable to projects implemented in programming languages other than Java. However, the 49 selected projects are popular and have been frequently used in other code metrics studies.

Although we intended to include the fan-in metric in RQ1, we were unable to do so due to the complexity of generating thousands of call graphs for each project. This challenge stemmed from the need to collect code metrics at different commits for the same software project. However, this limitation should not affect our conclusions, as SATD methods consistently showed significant differences from NOT-SATD methods across all other studied metrics. An exception in a single metric is unlikely to alter our overall findings.

Internal validity may be influenced by our choice of statistical tests: the Wilcoxon Rank-Sum and Cliff's Delta tests. However, both tests are widely used and well-established in software engineering research. We identified SATD through source code comments, but this approach has limitations. In some cases, developers may not include comments when introducing SATD (e.g., they may add a commit message acknowledging SATD but omit a corresponding comment), or they might write a comment about SATD, remove the problematic code, but fail to update or remove the comment, leading to inconsistencies between the code and the comments.

Construct validity is hampered by our selection of the CodeShovel tool for constructing method change history. Also, our accuracy in detecting SATD and their removal can be impacted by developers' forgetfulness in writing or removing SATD comments. We also used the SATD detector tool developed by Liu *et al.* [14]. A recent study [89] has shown the potential of the Large Language Models (LLMs) in detecting SATD with higher accuracy.

Conclusion validity of our study is influenced by the various threats outlined above.

VI. CONCLUSION

In conclusion, when method level granularity is considered, our study provides strong evidence that Self-Admitted Technical Debts have a significant negative impact on software quality and maintainability. We found that methods containing SATDs are larger, more complex, harder to maintain, and more prone to changes and bugs, ultimately increasing the burden of future maintenance. This highlights the importance of early detection and resolution of SATDs, as unresolved debts can deteriorate software quality over time.

For practitioners, our findings emphasize the need to prioritize SATD management, not only to improve code quality but also to reduce long-term maintenance costs. We also call for further research exploring different types of SATDs and their varying impacts, as well as the development of automated tools for accurate detection and remediation. Additionally, future studies should consider integrating SATD into machine learning models for predictive maintenance.

Overall, our study underscores the critical role SATDs play in software maintainability, providing a solid foundation for future work focused on better understanding and managing technical debt at the method level.

ACKNOWLEDGMENT

The authors would like to thank the Undergraduate Research Awards (URA, University of Manitoba) and the University Research Grants Program (URGP, University of Manitoba) for supporting this research. Also, we have used ChatGPT to improve the presentation (e.g., checking grammar and spelling) of this manuscript.

REFERENCES

- [1] D. Kafura and G. Reddy, "The use of software complexity metrics in software maintenance," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 3, pp. 335–343, 1987.
- [2] J. Börstler and B. Paech, "The role of method chains and comments in software readability and comprehension – an experiment," *IEEE Transactions on Software Engineering*, vol. 42, pp. 1–1, 09 2016.
- [3] M. Carr and C. Wagner, "A study of reasoning processes in software maintenance management," *Information Technology and Management*, vol. 3, pp. 181–203, 2002.
- [4] S. Chowdhury, G. Uddin, H. Hemmati, and R. Holmes, "Method-level bug prediction: Problems and promises," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 4, apr 2024.
- [5] L. Pascarella, F. Palomba, and A. Bacchelli, "On the performance of method-level bug prediction: A negative result," *Journal of Systems and Software*, vol. 161, p. 110487, 2020.
- [6] D. Romano and M. Pinzger, "Using source code metrics to predict change-prone java interfaces," in *2011 27th IEEE International Conference on Software Maintenance*, 2011, pp. 303–312.
- [7] D. Spadini, F. Palomba, A. Zaidman, M. Bruntink, and A. Bacchelli, "On the relation of test smells to software code quality," in *2018 IEEE International Conference on Software Maintenance and Evolution*, 2018.
- [8] Y. Zhou, B. Xu, and H. Leung, "On the ability of complexity metrics to predict fault-prone classes in object-oriented systems," *Journal of Systems and Software*, vol. 83, no. 4, pp. 660 – 674, 2010.
- [9] S. Chowdhury, R. Holmes, A. Zaidman, and R. Kazman, "Revisiting the debate: Are code metrics useful for measuring maintenance effort?" *Empirical Software Engineering (EMSE)*, vol. 27, no. 6, p. 31, 2022.
- [10] R. R. De Almeida, C. Treude, and U. Kulesza, "Tracy: A business-driven technical debt prioritization framework," in *2019 IEEE International Conference on Software Maintenance and Evolution*, 2019.
- [11] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Software Maintenance and Evolution (ICSME), 2014 IEEE International Conference on*. IEEE, 2014, pp. 91–100.
- [12] S. Esfandiari and A. Sami, "An exploratory study of the relationship between satd and other software development activities," in *2023 13th International Conference on Computer and Knowledge Engineering (ICCKE)*. IEEE, 2023, pp. 096–101.
- [13] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.
- [14] Z. Liu, Q. Huang, X. Xia, E. Shihab, D. Lo, and S. Li, "Satd detector: a text-mining-based self-admitted technical debt detection tool," in *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, 2018, p. 9–12.
- [15] X. Ren, Z. Xing, X. Xia, D. Lo, X. Wang, and J. Grundy, "Neural network-based detection of self-admitted technical debt: From performance to explainability," *ACM transactions on software engineering and methodology (TOSEM)*, vol. 28, no. 3, pp. 1–45, 2019.
- [16] E. d. S. Maldonado and E. Shihab, "Detecting and quantifying different types of self-admitted technical debt," in *2015 IEEE 7th international workshop on managing technical debt (MTD)*, 2015, pp. 9–15.
- [17] Y. Li, M. Soliman, and P. Avgeriou, "Automatic identification of self-admitted technical debt from four different sources," *Empirical Software Engineering*, vol. 28, no. 3, p. 65, 2023.
- [18] Z. Guo, S. Liu, J. Liu, Y. Li, L. Chen, H. Lu, and Y. Zhou, "How far have we progressed in identifying self-admitted technical debts? a comprehensive empirical study," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 4, pp. 1–56, 2021.
- [19] R. Sharma, R. Shahbazi, F. H. Fard, Z. Codabux, and M. Vidoni, "Self-admitted technical debt in r: detection and causes," *Automated Software Engineering*, vol. 29, no. 2, p. 53, 2022.
- [20] F. Zampetti, A. Serebrenik, and M. Di Penta, "Automatically learning patterns for self-admitted technical debt removal," in *2020 IEEE 27th International conference on software analysis, evolution and reengineering (SANER)*, 2020, pp. 355–366.
- [21] —, "Was self-admitted technical debt removal a real removal? an in-depth perspective," in *Proceedings of the 15th international conference on mining software repositories*, 2018, pp. 526–536.
- [22] E. D. S. Maldonado, R. Abdalkareem, E. Shihab, and A. Serebrenik, "An empirical study on the removal of self-admitted technical debt," in *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2017, pp. 238–248.
- [23] Y. Kashiwa, R. Nishikawa, Y. Kamei, M. Kondo, E. Shihab, R. Sato, and N. Ubayashi, "An empirical study on self-admitted technical debt in modern code review," *Information and Software Technology*, vol. 146, p. 106855, 2022.
- [24] Y. Li, M. Soliman, and P. Avgeriou, "Presti: Predicting repayment effort of self-admitted technical debt using textual information," *Available at SSRN 4724886*.
- [25] S. Wehaibi, E. Shihab, and L. Guerrouj, "Examining the impact of self-admitted technical debt on software quality," in *2016 IEEE 23rd international conference on software analysis, evolution, and reengineering (SANER)*, vol. 1, 2016, pp. 179–188.
- [26] G. Bavota and B. Russo, "A large-scale empirical study on self-admitted technical debt," in *Proceedings of the 13th international conference on mining software repositories*, 2016, pp. 315–326.
- [27] G. Sierra, E. Shihab, and Y. Kamei, "A survey of self-admitted technical debt," *Journal of Systems and Software*, vol. 152, pp. 70–82, 2019.
- [28] B. A. Kitchenham, T. Dyba, and M. Jorgensen, "Evidence-based software engineering," in *Proceedings. 26th International Conference on Software Engineering*, 2004, pp. 273–281.
- [29] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," in *Proceedings of the 36th International Conference on Software Engineering*, 2014, p. 435–445.
- [30] F. Grund, S. Chowdhury, N. C. Bradley, B. Hall, and R. Holmes, "Codeshovel: Constructing method-level source code histories," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2021, pp. 1510–1522.
- [31] Y. Gil and G. Lalouche, "On the correlation between size and metric validity," *Emp. Soft. Eng.*, vol. 22, no. 5, pp. 2585–2611, 2017.
- [32] K. El Emam, S. Benlarbi, N. Goel, and S. N. Rai, "The confounding effect of class size on the validity of object-oriented metrics," *IEEE Transactions on Software Engineering*, vol. 27, no. 7, pp. 630–650, 2001.
- [33] D. I. K. Sjøberg, A. Yamashita, B. C. D. Anda, A. Mockus, and T. Dybå, "Quantifying the effect of code smells on maintenance effort," *IEEE Transactions on Software Engineering*, vol. 39, no. 8, 2013.
- [34] D. Landman, A. Serebrenik, and J. Vinju, "Empirical analysis of the relationship between cc and sloc in a large corpus of java methods," in *IEEE International Conference on Software Maintenance and Evolution*, 2014, pp. 221–230.
- [35] M. Jodavi and N. Tsantalis, "Accurate method and variable tracking in commit history," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 183–195.
- [36] E. Maldonado, E. Shihab, and N. Tsantalis, "Using natural language processing to automatically detect self-admitted technical debt," *IEEE Transactions on Software Engineering*, 2017.
- [37] M. Iammarino, F. Zampetti, L. Aversano, and M. Di Penta, "An empirical study on the co-occurrence between refactoring actions and self-admitted technical debt removal," *Journal of Systems and Software*, vol. 178, p. 110976, 2021.
- [38] J. Liu, Q. Huang, X. Xia, E. Shihab, D. Lo, and S. Li, "An exploratory study on the introduction and removal of different types of technical debt in deep learning frameworks," *Empirical Software Engineering*, vol. 26, pp. 1–36, 2021.
- [39] A. Mastropaolo, M. Di Penta, and G. Bavota, "Towards automatically addressing self-admitted technical debt: How far are we?" in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 585–597.
- [40] F. Zampetti, C. Noiseux, G. Antoniol, F. Khomh, and M. Di Penta, "Recommending when design technical debt should be self-admitted," in *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2017, pp. 216–226.
- [41] Y. Shin, A. Meneely, L. Williams, and J. A. Osborne, "Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities," *IEEE Transactions on Software Engineering*, vol. 37, no. 6, pp. 772–787, 2011.
- [42] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [43] R. Mo, S. Wei, Q. Feng, and Z. Li, "An exploratory study of bug prediction at the method level," *Information and Software Technology*, vol. 144, p. 110616, 2022.
- [44] T. Shippey, T. Hall, S. Counsell, and D. Bowes, "So you need more method level datasets for your software defect prediction? voilà!" in *ESEM '16*, 2016.

- [45] R. Moser, W. Pedrycz, and G. Succi, "Analysis of the reliability of a subset of change metrics for defect prediction," in *Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, 2008, p. 309–311.
- [46] F. Khomh, M. D. Penta, Y.-G. Guéhéneuc, and G. Antoniol, "An exploratory study of the impact of antipatterns on class change- and fault-proneness," *Empirical software engineering : an international journal*, vol. 17, no. 3, pp. 243–275, 2012.
- [47] F. Palomba, M. Zanoni, F. A. Fontana, A. D. Lucia, and R. Oliveto, "Toward a smell-aware bug prediction model," in *IEEE Transactions on Software Engineering*, vol. 45, no. 2, 2017, pp. 194–218.
- [48] T. H. Ng, S.-C. Cheung, W. K. Chan, and Y.-T. Yu, "Toward effective deployment of design patterns for software extension: A case study," in *Proceedings of the 2006 international workshop on Software quality*, 2006, pp. 51–56.
- [49] P. Ralph and E. Tempero, "Construct validity in software engineering research and software metrics," in *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018 (Christchurch, New Zealand)*, 2018, pp. 13–23.
- [50] B. Ray, V. Hellendoorn, S. Godhane, Z. Tu, A. Baccchelli, and P. Devanbu, "On the "naturalness" of buggy code," in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, 2016.
- [51] Y. Higo, S. Hayashi, and S. Kusumoto, "On tracking java methods with git mechanisms," *JSS*, vol. 165, p. 110571, 2020.
- [52] S. I. Ahmad, S. Chowdhury, and R. Holmes, "Impact of methodological choices on the analysis of code metrics and maintenance," *Journal of Systems and Software*, 2024.
- [53] A. Bhatia, F. Khomh, B. Adams, and A. E. Hassan, "An empirical study of self-admitted technical debt in machine learning software," *arXiv preprint arXiv:2311.12019*, 2023.
- [54] N. M. Razali, Y. B. Wah *et al.*, "Power comparisons of shapiro-wilk, kolmogorov-smirnov, lilliefors and anderson-darling tests," *Journal of statistical modeling and analytics*, vol. 2, no. 1, pp. 21–33, 2011.
- [55] P. He, B. Li, X. Liu, J. Chen, and Y. Ma, "An empirical study on software defect prediction with a simplified metric set," *Information and Software Technology*, vol. 59, pp. 170–190, 2015.
- [56] Y. Chen, P. Li, J. Xu, S. Guo, R. Zhou, Y. Zhang, T. Wei, and L. Lu, "Savior: Towards bug-driven hybrid testing," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 1580–1596.
- [57] F. Pecorelli, G. Catolino, F. Ferrucci, A. De Lucia, and F. Palomba, "Testing of mobile applications in the wild: A large-scale empirical study on android apps," in *Proceedings of the 28th international conference on program comprehension*, pp. 296–307.
- [58] S. Chowdhury, S. Borle, S. Romansky, and A. Hindle, "Greenscaler: training software energy models with automatic test generation," *Empirical software engineering*, vol. 24, no. 4, pp. 1649–1692, 2019.
- [59] A. A. Bangash, H. Sahar, A. Hindle, and K. Ali, "On the time-based conclusion stability of cross-project defect prediction models," *Empirical Softw. Engg.*, vol. 25, no. 6, 2020.
- [60] S. A. Chowdhury, G. Uddin, and R. Holmes, "An empirical study on maintainable method size in java," in *19th International Conference on Mining Software Repositories*, 2022, p. 252–264.
- [61] S. Scalabrino, M. Linares-Vásquez, D. Poshyvanyk, and R. Oliveto, "Improving code readability models with textual features," in *IEEE 24th International Conference on Program Comprehension*, 2016, pp. 1–10.
- [62] R. P. L. Buse and W. R. Weimer, "Learning a metric for code readability," *IEEE Trans. Softw. Eng.*, vol. 36, no. 4, pp. 546–558, 2010.
- [63] D. Posnett, A. Hindle, and P. Devanbu, "A simpler model of software readability," in *Proceedings of the 8th Working Conference on Mining Software Repositories (MSR)*, 2011, pp. 73–82.
- [64] J. Börstler and B. Paech, "The role of method chains and comments in software readability and comprehension—an experiment," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 886–898, 2016.
- [65] J. Johnson, S. Lubo, N. Yedla, J. Aponte, and B. Sharif, "An empirical study assessing source code readability in comprehension," in *2019 IEEE International Conference on Software Maintenance and Evolution*, 2019, pp. 513–523.
- [66] C. L. McClure, "A model for program complexity analysis," in *Proceedings of the 3rd International Conference on Software Engineering*, 1978, p. 149–157.
- [67] A. Hindle, M. W. Godfrey, and R. C. Holt, "Reading beside the lines: Indentation as a proxy for complexity metric," in *16th IEEE International Conference on Program Comprehension*, 2008, pp. 133–142.
- [68] Y. Benjamini and D. Yekutieli, "The control of the false discovery rate in multiple testing under dependency," *Annals of statistics*, pp. 1165–1188, 2001.
- [69] B. Curtis, S. B. Sheppard, P. Milliman, M. A. Borst, and T. Love, "Measuring the psychological complexity of software maintenance tasks with the halstead and mccabe metrics," *IEEE Transactions on Software Engineering*, vol. SE-5, no. 2, pp. 96–104, 1979.
- [70] U. Tiwari and S. Kumar, "Cyclomatic complexity metric for component based software," *SIGSOFT Softw. Eng. Notes*, vol. 39, no. 1, p. 1–6, Feb. 2014.
- [71] M. Alfadel, A. Kobilica, and J. Hassine, "Evaluation of halstead and cyclomatic complexity metrics in measuring defect density," in *2017 9th IEEE-GCC Conference and Exhibition*, 2017, pp. 1–9.
- [72] E. J. Weyuker, "Evaluating software complexity measures," *IEEE Transactions on Software Engineering*, vol. 14, no. 9, pp. 1357–1365, 1988.
- [73] M. Shepperd, "A critique of cyclomatic complexity as a software metric," *Software Engineering Journal*, vol. 3, no. 2, pp. 30–36, 1988.
- [74] E.-M. Arvanitou, A. Ampatzoglou, A. Chatzigeorgiou, and P. Avgeriou, "A method for assessing class change proneness," in *Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering*, 2017, pp. 186–195.
- [75] F. Khomh, M. Di Penta, and Y.-G. Gueheneuc, "An exploratory study of the impact of code smells on software change-proneness," in *2009 16th Working Conference on Reverse Engineering*, 2009, pp. 75–84.
- [76] G. Catolino, F. Palomba, A. De Lucia, F. Ferrucci, and A. Zaidman, "Enhancing change prediction models using developer-related factors," *Journal of Systems and Software*, vol. 143, pp. 14–28, 2018.
- [77] G. Catolino, F. Palomba, F. A. Fontana, A. De Lucia, A. Zaidman, and F. Ferrucci, "Improving change prediction models with code smell-related information," *Empirical Software Engineering*, vol. 25, pp. 49–95, 2020.
- [78] V. Antinyan, M. Staron, W. Meding, P. Österström, E. Wikström, J. Wranger, A. Henriksson, and J. Hansson, "Identifying risky areas of software code in agile/lean software development: An industrial experience report," in *IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering*, 2014, pp. 154–163.
- [79] A. Monden, D. Nakae, T. Kamiya, S. Sato, and K. Matsumoto, "Software quality analysis by code clones in industrial legacy software," in *Proceedings IEEE Symposium on Software Metrics*, 2002, pp. 87–94.
- [80] V. Antinyan, M. Staron, J. Derehag, M. Runsten, E. Wikström, W. Meding, A. Henriksson, and J. Hansson, "Identifying complex functions: By investigating various aspects of code complexity," in *2015 Science and Information Conference (SAI)*, 2015, pp. 879–888.
- [81] I. Scholtes, P. Mavrodiev, and F. Schweitzer, "From aristotle to ringelmann: a large-scale analysis of team productivity and coordination in open source software projects," *Empirical Software Engineering (EMSE)*, vol. 21, no. 2, pp. 642–683, 2016.
- [82] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet physics doklady*, vol. 10, 1966.
- [83] D. Ståhl, A. Martini, and T. Mårtensson, "Big bangs and small pops: On critical cyclomatic complexity and developer integration behavior," in *2019 IEEE/ACM 41st International Conference on Software Engineering: (ICSE-SEIP)*, 2019, pp. 81–90.
- [84] S. Scalabrino, G. Bavota, C. Vendome, M. Linares-Vásquez, D. Poshyvanyk, and R. Oliveto, "Automatically assessing code understandability: How far are we?" in *32nd IEEE/ACM International Conference on Automated Software Engineering*, 2017, pp. 417–427.
- [85] H. Kirinuki, Y. Higo, K. Hotta, and S. Kusumoto, "Hey! are you committing tangled changes?" in *Proceedings of the 22nd International Conference on Program Comprehension*, 2014, pp. 262–265.
- [86] K. Herzig, S. Just, and A. Zeller, "The impact of tangled code changes on defect prediction models," *Empirical Software Engineering*, vol. 21, no. 2, pp. 303–336, 2016.
- [87] A. Mocku and L. G. Votta, "Identifying reasons for software changes using historic databases," in *Proceedings 2000 International Conference on Software Maintenance*, 2000, pp. 120–130.
- [88] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, "Evaluating szz implementations through a developer-informed oracle," in *Proceedings of the 43rd International Conference on Software Engineering*, 2021, p. 436–447.
- [89] M. S. Sheikhaei, Y. Tian, S. Wang, and B. Xu, "An empirical study on the effectiveness of large language models for satd identification and classification," *Empirical Software Engineering*, vol. 29, no. 6, p. 159, 2024.